

hw__1

January 6, 2026

0.1 Import delle Librerie

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib
matplotlib.use("TkAgg")
```

0.2 Metodo della Bisezione (senza condizione Teorema degli Zeri)

```
[2]: def bisezione (fun,a,b,tol,maxit):
    fun_history = np.zeros(maxit)

    c=a
    for i in range (maxit):
        c=(a+b)/2
        if (fun(a)*fun(c)) > 0:
            a=c
        else:
            b=c
        fun_history[i]=c
        c_final=(a+b)/2
        if (b-a) / 2 < tol:
            break
    return (c_final, i, fun_history)
```

0.3 Metodo del Punto Fisso

```
[3]: def punto_fisso(g, fun, x_0, tol_1, tol_2, maxit) :

    fun_history = np.zeros(maxit)
    x_new=0

    for i in range (maxit):
        if np.abs(fun(x_0)) < tol_1:
            break
        x_new = g(x_0)

        if (np.abs(x_0-x_new)) < tol_2:
```

```

        break
    fun_history[i] = x_new
    x_0 = x_new
    return x_new, i, fun_history

```

0.4 Metodo di Newton

```

[4]: def newton(f,df,x_0,tol_1,tol_2,maxit):
    fun_history = np.zeros(maxit)
    x_new=0
    for i in range (maxit):
        if np.abs(f(x_0)) < tol_1:
            break
        x_new=x_0-f(x_0)/df(x_0)
        if np.abs(x_0-x_new) < tol_2:
            break
        fun_history[i]=x_new
        x_0=x_new
    return x_0, i, fun_history

```

0.4.1 Generatore di Grafici:

```

[5]: def graph_generator(x, f, g, path_bisezione, path_newton, path_fix, titolo_f,
    ↪titolo_g):

    plt.figure(figsize=(14,6))

    plt.subplot(1,2,1)
    plt.title("Ricerca dello zero")
    plt.plot(x, f(x), label=f"{titolo_f}", color='black')
    plt.grid(linestyle=":")
    plt.axhline(0, color='red', linestyle='--', label='y = 0', alpha=0.5)

    if path_bisezione is not None and len(path_bisezione) > 0:
        y_bis = f(path_bisezione)
        plt.plot(path_bisezione, y_bis, 'x', color='green', markersize=8,
    ↪label='Bisezione (iter)')
        plt.plot(path_bisezione[0], y_bis[0], 'o', color='green',
    ↪fillstyle='none', label='Start Bisezione')
    else:
        print("Grafico: Nessun dato per Bisezione (o metodo fallito).")

    if path_newton is not None and len(path_newton) > 0:
        y_new = f(path_newton)
        plt.plot(path_newton, y_new, '-.', color='blue', markersize=6,
    ↪label='Newton (path)', alpha=0.6)

```

```

plt.plot(path_newton[0], y_new[0], 'o', color='blue', fillstyle='none',
↪label='Start Newton')
    else:
        print("Grafico: Nessun dato per Newton (o metodo fallito).")

plt.legend()

plt.subplot(1,2,2)
plt.title("Metodo del punto fisso")
plt.plot(x, g(x), label=f"{titolo_g}", color='black')
plt.plot(x, x, label="y=x")
plt.grid(linestyle=":")
plt.axhline(0, color='red', linestyle='--', label='y = 0', alpha=0.5)

if path_fix is not None and len(path_fix) > 0:
    y_fix = g(path_fix)
    plt.plot(path_fix, y_fix, 'x', color='magenta', label='Valori g(x)')
    plt.plot(path_fix[0], y_fix[0], 'o', color='magenta', fillstyle='none',
↪label='Start Punto Fisso')
    else:
        print("Grafico: Nessun dato per Punto Fisso (o metodo fallito).")

plt.legend()

plt.show()

```

0.4.2 Stampare i Risultati:

```

[6]: def get_result(f1, df1, g1, TOL_1, TOL_2, MAX_IT, x0, a, b):

    print("<-- METODO DELLA BISEZIONE -->")
    sol_bi, ite_bi, path_bi = bisezione(f1, a, b, TOL_1, MAX_IT)
    print(f"Soluzione: {sol_bi}")
    print(f"Iterazioni: {ite_bi}")

    print("<-- METODO DEL PUNTO FISSO -->")
    sol_middle, ite_middle, path_middle = punto_fisso(g1, f1, x0, TOL_1, TOL_2,
↪MAX_IT)
    print(f"Soluzione: {sol_middle}")
    print(f"Iterazioni: {ite_middle}")

    print("<-- METODO DI NEWTON -->")
    sol_new, ite_new, path_new = newton(f1, df1, x0, TOL_1, TOL_2, MAX_IT)
    print(f"Soluzione: {sol_new}")
    print(f"Iterazioni: {ite_new}")

```

```
return path_bi, path_middle, path_new
```

```
[7]: TOL_1 = 1.e-6
      TOL_2 = 1.e-6
      MAX_IT = 100
      n=200
```

0.5 Caso di Studio 1: $f(x) = \ln(x+1) - x$

- $g(x) = \ln(x+1)$ per il punto fisso.
- $f'(x) = \frac{1}{x+1} - 1$ per il metodo di Newton.

```
[8]: f1 = lambda x: np.log(x+1) - x
      g1 = lambda x: np.log(x+1)
      df1 = lambda x: 1/(x+1)-1

      a1 = 0
      b1 = 1.0

      x1 = np.linspace(a1, b1, n)
      x_start1 = 0.8

      title1_f = r"$f(x) = \ln(x+1) - x$"
      title1_g = r"$g(x) = \ln(x+1)$"

      path_bi_1, path_mid_1, path_new_1 = get_result(f1, df1, g1, TOL_1, TOL_2,
      ↪MAX_IT, x_start1, a1, b1)
      graph_generator(x1, f1, g1, path_bi_1, path_mid_1, path_new_1, title1_f,
      ↪title1_g)

<-- METODO DELLA BISEZIONE -->
Soluzione: 9.5367431640625e-07
Iterazioni: 18
<-- METODO DEL PUNTO FISSO -->
Soluzione: 0.0197459513380617
Iterazioni: 99
<-- METODO DI NEWTON -->
Soluzione: 0.001043352587292232
Iterazioni: 9
```

0.6 Caso di Studio 2: $f(x) = x^2 - \cos(x)$

- $g(x) = \sqrt{\cos(x)}$ per il metodo del punto fisso
- $f'(x) = 2x + \sin(x)$ per il metodo di Newton.

```
[9]: f2 = lambda x: x**2 - np.cos(x)
g2 = lambda x: np.sqrt(np.cos(x))
df2 = lambda x: 2*x + np.sin(x)

a2 = 0
b2 = np.pi/2

x2 = np.linspace(a2, b2, n)
x_start2 = 0.8

title2_f = r"$f(x) = x^2 - \cos(x)$"
title2_g = r"$g(x) = \sqrt{\cos(x)}$"

path_bi_2, path_mid_2, path_new_2 = get_result(f2, df2, g2, TOL_1, TOL_2,
↪MAX_IT, x_start2, a2, b2)
graph_generator(x2, f2, g2, path_bi_2, path_mid_2, path_new_2, title1_f,
↪title1_g)

<-- METODO DELLA BISEZIONE -->
Soluzione: 0.8241319273056174
Iterazioni: 19
<-- METODO DEL PUNTO FISSO -->
Soluzione: 0.8241320250620604
Iterazioni: 13
<-- METODO DI NEWTON -->
Soluzione: 0.8241323765632924
Iterazioni: 2
```

0.7 Caso di Studio 3: $\sin(x) - \frac{x}{2}$

- $f'(x) = \cos(x) - \frac{1}{2}$
- $g(x) = 2 \sin(x)$

```
[10]: f3 = lambda x: np.sin(x) - x/2
g3 = lambda x: 2*np.sin(x)
df3 = lambda x: np.cos(x) - 1/2

a3 = -2
b3 = 2

x3 = np.linspace(a3, b3, n)

x_start3 = 0.8

title3_f = r"$f(x) = \sin(x) - \frac{x}{2}$"
title3_g = r"$g(x) = 2\sin(x)$"
```

```
path_bi_3, path_mid_3, path_new_3 = get_result(f3, df3, g3, TOL_1, TOL_2,
↪MAX_IT, x_start3, a3, b3)
graph_generator(x3, f3, g3, path_bi_3, path_mid_3, path_new_3, title3_f,
↪title3_g)
```

```
<-- METODO DELLA BISEZIONE -->
Soluzione: -1.895493507385254
Iterazioni: 20
<-- METODO DEL PUNTO FISSO -->
Soluzione: 1.895493071666415
Iterazioni: 27
<-- METODO DI NEWTON -->
Soluzione: -1.8954942678892686
Iterazioni: 6
```

0.8 Caso di Studio 4: $e^x - 3x$

- $f'(x) = e^x - 3$
- $g(x) = \frac{e^x}{3}$

```
[11]: f4 = lambda x: np.e**x-3*x
g4 = lambda x: 1/3*np.e**x
df4 = lambda x: np.e**x-3

a4 = -1
b4 = 2

x4 = np.linspace(a4, b4, n)

x_start4 = 0.8

title4_f = r"$f(x) = \sin(x) - \frac{x}{2}$"
title4_g = r"$g(x) = 2\sin(x)$"

path_bi_4, path_mid_4, path_new_4 = get_result(f4, df4, g4, TOL_1, TOL_2,
↪MAX_IT, x_start4, a4, b4)
graph_generator(x4, f4, g4, path_bi_4, path_mid_4, path_new_4, title4_f,
↪title4_g)
```

```
<-- METODO DELLA BISEZIONE -->
Soluzione: 0.619060754776001
Iterazioni: 20
<-- METODO DEL PUNTO FISSO -->
Soluzione: 0.6190627522440586
Iterazioni: 24
<-- METODO DI NEWTON -->
```

Soluzione: 0.6190612867336015
Iterazioni: 4